

Curso: Computação	Turno: Matutino	Semestre: 2º
Disciplina: Inteligência Artificial	Professor: Rafael T. Anchiêta	Data: 24/08/26
Nome:		

Agentes Inteligentes

1 Visão geral

Nesta atividade, você desenvolverá agentes para um ambiente de resgate representado por uma grade. Toda a infraestrutura do ambiente, os cenários, o simulador, a coleta de métricas e os scripts de experimentação são fornecidos.

Você deverá implementar **apenas as classes dos agentes** presentes em:

`src/agents/student_agents.py`

Não é necessário modificar o ambiente, o simulador ou os scripts de avaliação.

As arquiteturas **obrigatórias** são:

1. agente reativo simples;
2. agente reativo baseado em modelo;
3. agente de aprendizado.

Os agentes baseados em objetivos e em utilidade são **opcionais**. Sua implementação adequada normalmente utiliza algoritmos de busca e planejamento, conteúdos que serão estudados posteriormente.

2 Problema

Um robô atua em um edifício após um acidente. O ambiente é uma grade composta por células.

Símbolo	Significado
.	espaço livre
#	obstáculo
V	vítima
H	região perigosa
C	estação de recarga
E	saída
R	posição atual do robô

O robô deve resgatar o maior número possível de vítimas, conservar energia, evitar regiões perigosas e, idealmente, terminar a missão na saída.

O conjunto de ações é:

$$A = \{NORTE, SUL, LESTE, OESTE, RESGATAR, RECARREGAR, ESPERAR\}.$$

3 Percepção e interface

O agente **não tem acesso direto ao objeto do ambiente**. A única entrada de `act()` é uma instância de `Perception`.

A percepção contém:

- passo atual;
- posição do agente;
- nível atual e máximo de bateria;
- células atualmente visíveis;
- indicação de vítima, carregador ou saída na posição atual;
- número de vítimas já resgatadas;
- número total de vítimas do episódio.

Todos os agentes herdam da interface:

```
class Agent(ABC):
    @abstractmethod
    def act(self, perception: Perception) -> Action:
        ...

    def observe_transition(self, transition: Transition) -> None:
        pass

    def reset(self) -> None:
        pass
```

Em cenários parcialmente observáveis, o agente vê apenas uma vizinhança local. Nesse caso,

$$o_t \neq s_t,$$

onde o_t é a observação e s_t o estado completo do ambiente.

4 Ambientes fornecidos

Quatro cenários são disponibilizados e não devem ser alterados.

Cenário 1 — simple

Ambiente determinístico e completamente observável, com bateria abundante e poucos obstáculos. Serve para depuração inicial.

Cenário 2 — partial

Ambiente determinístico e parcialmente observável. O agente observa apenas a vizinhança próxima. Esse cenário evidencia a diferença entre

$$a_t = f(o_t)$$

e

$$a_t = f(o_t, \hat{s}_t),$$

onde $\hat{s}_t$ representa o estado interno do agente.

Cenário 3 — risky

Ambiente parcialmente observável com regiões perigosas e bateria mais restrita. Entrar em uma célula perigosa aumenta o consumo de energia e gera penalidade.

Cenário 4 — stochastic

Além das características anteriores, movimentos podem falhar com probabilidade

$$P(\text{falha}) = 0,15.$$

Consequentemente,

$$P(s_{t+1} \mid s_t, a_t)$$

não está necessariamente concentrada em um único próximo estado.

5 Medida de desempenho

O simulador utiliza recompensas e penalidades. Entre elas:

- +100 por vítima resgatada;
- custo por passo;
- penalidade por entrar em região perigosa;
- penalidade por ações inválidas;
- penalidade elevada se a bateria chegar a zero;
- bônus de missão quando todas as vítimas são resgatadas e o agente alcança a saída.

Não modifique a função de recompensa, os mapas ou os limites de bateria para beneficiar uma arquitetura específica.

6 Estrutura do projeto

```
atividade_agentes/  
|  
|-- atividade_agentes.pdf  
|-- README.md  
|-- requirements.txt  
|  
|-- src/  
|   |-- environment.py  
|   |-- scenarios.py  
|   |-- simulator.py  
|   |-- run_episode.py  
|   |-- run_experiments.py  
|   |-- plot_results.py  
|   |-- agent_factory.py  
|   |-- types.py  
|   |  
|   -- agents/  
|       |-- base.py  
|       |-- baseline.py  
|       -- student_agents.py  
|  
-- tests/  
    -- check_submission.py
```

7 Preparação

Crie um ambiente virtual e instale as dependências:

```
python -m venv .venv
source .venv/bin/activate
pip install -r requirements.txt
```

No Windows:

```
.venv\Scripts\activate
```

Antes de implementar qualquer agente, teste a infraestrutura:

```
python -m src.run_episode --agent random --scenario simple --render
```

O agente `random` é fornecido apenas como baseline de infraestrutura.

8 Implementação dos agentes

1. Implemente:

```
class SimpleReflexAgent(Agent):
    ...
```

A decisão deve depender **exclusivamente da percepção atual**:

$$a_t = f(o_t).$$

Restrições

O agente reativo simples não deve utilizar:

- mapa interno;
- histórico de percepções;
- conjunto de células visitadas;
- contadores de visita;
- Q-table;
- qualquer outro estado persistente do episódio.

Você pode construir regras condição–ação, por exemplo:

- se houver vítima na célula atual, resgatar;
- se a bateria estiver baixa e o agente estiver no carregador, recarregar;
- evitar uma parede atualmente visível;
- selecionar uma direção usando apenas a observação corrente.

>>> Explique por que um agente puramente reflexivo pode entrar em ciclos, como

$$A \rightarrow B \rightarrow A \rightarrow B \rightarrow \dots$$

quando não dispõe de memória.

2. Implemente:

```
class ModelBasedAgent(Agent):  
    ...
```

Agora o agente pode manter um estado interno:

$$\hat{s}_t = \text{Update}(\hat{s}_{t-1}, a_{t-1}, o_t).$$

Sugestões de informações que podem ser armazenadas:

- células já observadas;
- células visitadas;
- número de visitas a cada posição;
- obstáculos e perigos conhecidos;
- última ação.

Nesta atividade, **não é obrigatório usar BFS, DFS, Dijkstra, A* ou qualquer outro algoritmo de busca**. Uma estratégia local que favoreça células menos visitadas é suficiente para estudar o papel da memória.

>>> Quando memória melhora o comportamento de um agente?

3. Implemente:

```
class LearningAgent(Agent):  
    ...
```

A sugestão é utilizar Q-learning tabular. A regra de atualização é

$$Q(s, a) \leftarrow Q(s, a) + \alpha \left[r + \gamma \max_{a'} Q(s', a') - Q(s, a) \right]$$

Se o episódio tiver terminado, o termo futuro deve ser zero.

A política deverá utilizar exploração ϵ -greedy:

$$a = \begin{cases} \text{ação aleatória,} & \text{com probabilidade } \epsilon, \\ \arg \max_a Q(s, a), & \text{caso contrário} \end{cases}$$

Codificação do estado

Você deverá definir uma representação suficientemente compacta para Q-learning tabular. Uma possibilidade é

$$s = (x, y, b, n, s, l, o, v, c),$$

em que x, y são a posição, b é a bateria discretizada, n, s, l, o descrevem a vizinhança e v, c indicam vítima e carregador.

Não é obrigatório utilizar exatamente essa representação.

Persistência entre episódios

A Q-table deve **sobreviver entre episódios**. Portanto, `reset()` não deve apagar os valores aprendidos.

9 Execução individual

Após implementar cada agente, execute:

```
python -m src.run_episode --agent simple --scenario simple --render
python -m src.run_episode --agent model --scenario partial --render
python -m src.run_episode --agent learning --scenario stochastic --render
```

Para verificar a interface:

```
python -m tests.check_submission
```

10 Experimento obrigatório

Os agentes devem ser comparados nos mesmos cenários e com as mesmas sementes aleatórias:

```
python -m src.run_experiments \
    --agents simple model learning \
    --scenarios simple partial risky stochastic \
    --episodes 100 \
    --output results.csv
```

O arquivo `results.csv` contém:

- recompensa total;
- número de passos;
- vítimas resgatadas;
- energia consumida;
- entradas em regiões perigosas;
- sucesso da missão;
- bateria ao final do episódio.

10.1 Gráficos

Execute:

```
python -m src.plot_results --input results.csv --output-dir plots
```

O script gera gráficos comparativos das principais métricas e uma curva de aprendizagem.

11 Análise experimental

Construa uma tabela com resultados agregados:

Agente	Recompensa	Passos	Resgatadas	Energia	Sucesso
Reativo simples	–	–	–	–	–
Baseado em modelo	–	–	–	–	–
Aprendizado	–	–	–	–	–

Apresente, no mínimo:

1. média da recompensa;
2. desvio-padrão da recompensa;
3. taxa de sucesso;

4. média de vítimas resgatadas;
5. consumo médio de energia.

Para uma métrica R :

$$\bar{R} = \frac{1}{N} \sum_{i=1}^N R_i,$$
$$\sigma_R = \sqrt{\frac{1}{N} \sum_{i=1}^N (R_i - \bar{R})^2}.$$

12 Experimentos com o agente de aprendizado

Realize pelo menos três configurações de exploração:

$$\epsilon \in \{0,01, 0,10, 0,30\}.$$

Compare:

- velocidade de aprendizado;
- estabilidade;
- recompensa final;
- quantidade de estados explorados.

Discuta o conflito entre

exploração e exploração.

13 Questões para entendimento

1. Em qual cenário o agente reativo simples apresentou melhor desempenho? Por quê?
2. Em quais condições o estado interno melhorou o desempenho?
3. O agente baseado em modelo sempre superou o agente reativo simples? Explique.
4. Qual foi o principal efeito da observabilidade parcial?
5. Qual foi o principal efeito da estocasticidade?
6. Como a escolha de ϵ modificou o comportamento do agente de aprendizado?
7. O agente de aprendizado melhorou ao longo dos episódios? Apresente evidências quantitativas.
8. Qual arquitetura apresentou melhor relação entre complexidade e desempenho?
9. Um agente arquiteturalmente mais complexo necessariamente apresenta melhor desempenho? Justifique empiricamente.
10. Que mudanças no ambiente fariam um agente baseado em objetivos tornar-se claramente necessário?

14 Entregáveis

A entrega deverá conter:

1. `src/agents/student_agents.py`;
2. `results.csv`;
3. os gráficos produzidos;
4. instruções adicionais necessárias para reproduzir os experimentos, caso existam.